nature communications

Article

https://doi.org/10.1038/s41467-026-77407-1

Unified down-stream analysis of crosslinking mass spectrometry results with pyXLMS

Received: 7 January 2026

Accepted: 20 August 2026

Published online: 04 September 2026

Check for updates

Micha J. Birklbauer[1,2] ✉, Louise M. Buur[1,2], Sabrina Kaser[1], Fränze Müller[3], Manuel Matzinger[3], Karl Mechtler[3,4,5], Stephan Winkler[1,2] & Viktoria Dorfer[1] ✉

Crosslinking mass spectrometry has become the method of choice for the identification of protein-protein interactions and for gaining insight into the structures of proteins in vivo. However, connecting crosslink search engine results with down-stream analysis tools, and therefore gaining biological insight from crosslink identifications, has remained a manual and cumbersome step in the analysis that often requires expert bioinformatics knowledge. Here we introduce pyXLMS, a python package and public web application which aims to simplify and streamline this intermediate step, enabling researchers even without bioinformatics knowledge to conduct in-depth crosslink analyses. In its current state pyXLMS supports input from more than seven different crosslink search engines, as well as the mzIdentML format of the HUPO Proteomics Standards Initiative. Data processing and quality control is facilitated by functionality that is directly available within pyXLMS such as aggregation, validation, annotation, filtering, and visualization. In addition, the data can easily be exported to more than ten supported down-stream analysis tools and formats. We demonstrate the applicability and benefits of pyXLMS by re-analyzing a publicly available crosslink dataset with a variety of different search engines and show how the same data analysis workflow can be applied using pyXLMS.

Crosslinking mass spectrometry (XLMS) has matured into a powerful tool for structural, molecular, and systems biology[1] and has become the method of choice for gaining insight into the structures of proteins in vivo[2,3] and the identification of protein-protein interactions[4,5] potentially up to system-wide scale. Advancements and optimizations in crosslinking chemistry[6–8], experimental procedures[9–11], instrumentation[12], and the release of numerous robust crosslink search engines[13–15] have made crosslink identification increasingly more appealing even for complex samples and proteome-wide searches. Plenty of comprehensive reviews have been published in recent years highlighting successful XLMS applications, potential pitfalls and drawbacks[16–22]. However, while XLMS has seen substantial progress and become more and more of a routine problem with established workflows, connecting crosslink search engine results with down-stream analysis tools such as AlphaLink2[23], ProXL[24,25], xiVIEW[26], or visualization tools like PyMOL[27] and ChimeraX[28–30] has remained a manual and cumbersome step in the analysis. It often requires extensive data re-shaping or transformation that is only possible for scientists with a strong bioinformatics background and yet this step is crucial for gaining biological insight from the identified crosslinks.

[1]Bioinformatics Research Group, University of Applied Sciences Upper Austria, Hagenberg, Upper Austria, Austria. [2]Institute for Symbolic Artificial Intelligence, Johannes Kepler University Linz, Linz, Upper Austria, Austria. [3]Institute of Molecular Pathology (IMP), Vienna BioCenter (VBC), Vienna, Vienna, Austria. [4]Institute of Molecular Biotechnology (IMBA), Austrian Academy of Sciences, Vienna BioCenter (VBC), Vienna, Vienna, Austria. [5]Gregor Mendel Institute (GMI), Austrian Academy of Sciences, Vienna BioCenter (VBC), Vienna, Vienna, Austria.
✉e-mail: micha.birklbauer@fh-hagenberg.at; viktoria.dorfer@fh-hagenberg.at

Even though the mzIdentML format[31] developed by the HUPO Proteomics Standards Initiative has introduced support for crosslinks already in version 1.2[32] in 2017 and further extended the format specifically for the reporting of crosslink results recently in version 1.3[33], many crosslink search engines and down-stream analysis tools have yet to adopt the format. Most crosslink search engines report results at different aggregation levels, for example at the crosslink-spectrum-match (CSM), crosslink/peptide pair/residue pair (XL), or protein-protein interaction (PPI) level, in most cases in some kind of tabular format. This is similar for down-stream analysis tools that usually require one of the aggregation levels, in most cases either CSM or residue pair level, also mostly in tabular formats for further processing. Although some down-stream analysis tools also implement their own parsers for reports of certain crosslink search engines, researchers are mostly left to their own accords when it comes to connecting crosslink identifications from search engines to down-stream analysis tools. A process that is not only time-consuming and requires bioinformatic knowledge, but also error prone.

In the following we present pyXLMS, a python package and web application with graphical user interface that aims to simplify and streamline this intermediate step, enabling scientists even without bioinformatics knowledge to conduct in-depth crosslink analyses and shifting the focus from data transformation to data interpretation. In its current state pyXLMS supports input from several different crosslink search engines and tools, including MaxLynx[15] (part of MaxQuant[34]), MeroX[35,36], MS Annika[13,37,38], pLink 2 and pLink 3[39], Scout[14], xiSearch[40] and the validation tool xiFDR[41], xiNET[42], xiVIEW[26], XlinkX[43], as well as the mzIdentML format[31–33] of the HUPO Proteomics Standards Initiative. Moreover, pyXLMS also supports a well-documented and human-readable custom tabular format that allows for parsing of any generic table. Data processing and quality control is facilitated by functionality that is directly available within pyXLMS such as aggregation, validation, annotation, filtering, and visualization of CSMs and XLs. pyXLMS focuses on data processing at the CSM and XL level with PPIs only being indirectly supported and derived via export to other down-stream analysis tools. Most importantly, pyXLMS enables easy export to the required data format of various popular and commonly used down-stream analysis tools such as AlphaLink2[23], ProXL[24,25], xiNET[42], xiVIEW[26], xiFDR[41], XlinkDB[44–47], xlms-tools[48], PyMOL[27](via PyXlinkViewer[49]), ChimeraX[28–30](via XMAS[50]), or IMP-X-FDR[51].

We demonstrate the applicability and benefits of pyXLMS by re-analyzing a publicly available crosslink dataset and show how the same data analysis workflow can be applied across several different crosslink search engines using pyXLMS, eliminating the need for manual adoption to the various data formats. pyXLMS is publicly available under a permissive open-source license via https://github.com/hgb-bin-proteomics/pyXLMS and welcoming contributions by the community with the aim of supporting as many crosslink search engines and down-stream analysis tools as possible. We are also hosting a publicly available and free of charge web server that includes most functionality at https://hgb-bin-proteomics.github.io/pyXLMS-app.

Results

pyXLMS—a python package to streamline crosslinking data analysis

Down-stream analysis of crosslinking results and therefore gaining meaningful biological insights, requires the connection of crosslink identifications from crosslink search engines with down-stream analysis tools. This process often involves extensive re-shaping, filtering and transformation of the originally reported data of the crosslink search engine, and an in-depth understanding of the chosen down-stream analysis tools is needed to successfully make this connection. We here present pyXLMS, a python package that we implemented to eliminate this intermediate process of connecting crosslink search engines with down-stream analysis tools. An overview of the package is given in Fig. 1.

pyXLMS handles reading of crosslink search engine results at both crosslink-spectrum-match (CSM) level and crosslink/peptide pair/residue pair (XL) level and currently supports several different crosslink search engines and tools as well as the mzIdentML format[31–33] as input. Furthermore, pyXLMS is generally able to read any crosslink output file even of non-supported crosslink search engines with minimal additional user input. It should be explicitly noted here that pyXLMS focuses on reading, processing, and exporting data on CSM and XL aggregation levels and does not support protein-protein interaction (PPI) level data, neither in reading nor processing. This has been an intentional design decision as the number of crosslink search engines supporting PPI level results is still limited and currently none of the down-stream analysis tools integrated with pyXLMS require PPI input. A full list of all supported crosslink search engines, tools and formats is given in Table 1.

Subsequently, pyXLMS allows direct inspection of the crosslinking data via a summary function that calculates common statistics, as well as several pre-defined visualization options that display score distribution, number of target and decoy matches, intra- and inter-protein matches (pyXLMS considers homomeric interlinks to be intralinks), and the most frequently crosslinked proteins and peptides. Data can then be filtered by target-decoy type, crosslink type, proteins of interest, peptide pair, or residue pair. One of the most important features in pyXLMS is the ability to re-annotate protein crosslink positions based on the given crosslinked peptides. Many crosslink search engines only report a single matching protein per peptide, even for ambiguous peptides that are shared among multiple proteins. This inherently leads to an incomplete list of crosslinked protein positions, which is crucial for calculating protein-protein interaction networks and understanding biological processes. The re-annotation functionality in pyXLMS matches all crosslinked peptides to the protein sequences of a given FASTA file and calculates all possible matches and protein crosslink positions, resolving this issue. pyXLMS also supports (re-)assignment of target-decoy labels which is especially useful when custom decoy or entrapment databases are used. Furthermore, pyXLMS offers several quality control functions such as target-decoy score distributions, comparison of the number of intra- against inter-protein links at both the result and single protein level, and peptide pair/residue pair coverage by CSMs (as shown in Supplementary Fig. 1). pyXLMS also integrates with STRING[52] and offers STRING score annotation of inter-protein links as well as two different visualization options of inter-link STRING scores (as shown in Supplementary Fig. 2), allowing assessment of how well inter-links match with known interactions and how many potentially new interactions were identified via crosslinking. Moreover, pyXLMS is able to deduplicate and filter CSMs and XLs to only contain unique matches, something that is desired for almost all down-stream analysis methods and required to make results comparable. Uniqueness becomes increasingly more important especially when combining results from different replicates or even different datasets altogether where duplicates are for certain. In pyXLMS uniqueness for CSMs is determined by the associated spectrum file and scan number while uniqueness of XLs is determined either by peptide sequence and peptide crosslink position (= unique peptide pairs), or by protein crosslink position (= unique residue pairs), depending on user choice. pyXLMS also supports aggregation of CSMs to XLs as well as validation via false-discovery-rate (FDR) estimation. However, validation is limited to results from crosslink search engines that report both target and decoy matches, as well as a single score for FDR estimation. This is currently only the case for MaxLynx, MS Annika, and xiSearch. Other crosslink search engines either do not report target-decoy information (e.g. pLink) or do not use a single score for FDR estimation (e.g. Scout).

https://doi.org/10.1038/s41467-026-77407-1

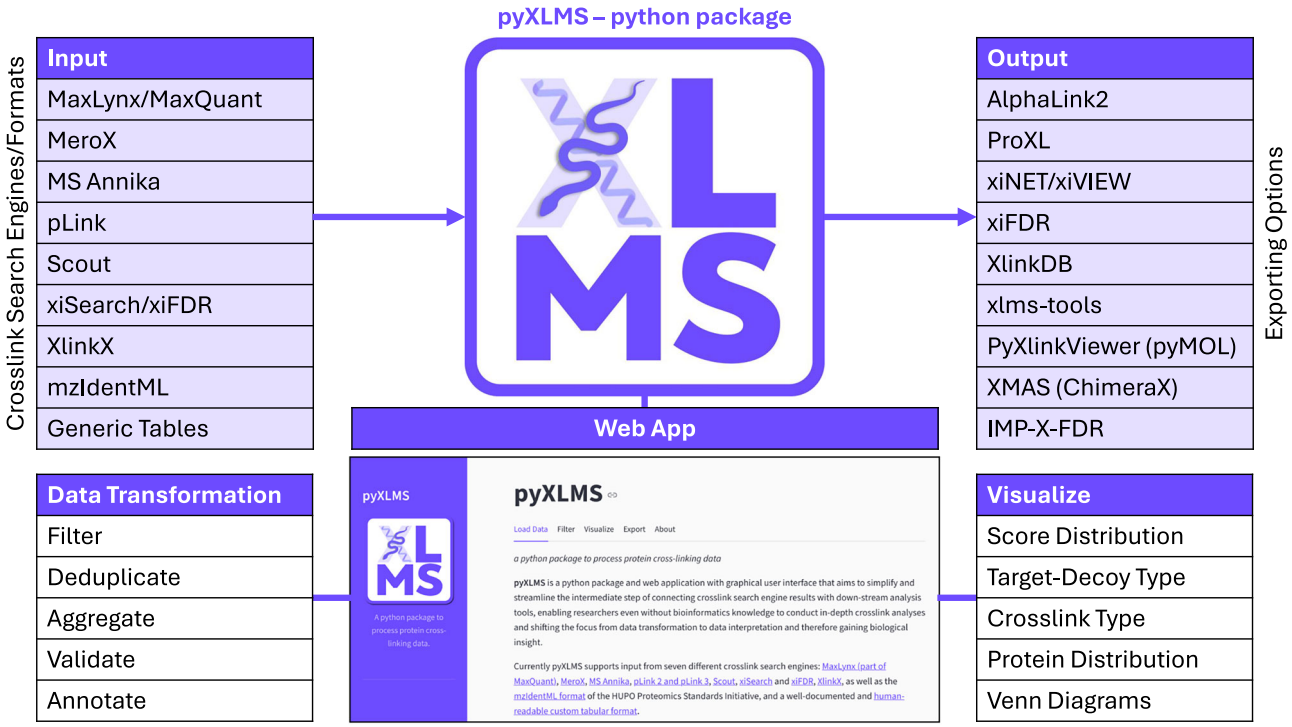


Fig. 1 | Overview of the pyXLMS python package. pyXLMS currently supports input from more than seven different crosslink search engines and tools as well as the mzIdentML format. Crosslinking data can then be transformed via different filtering, annotation, and validation options or visualized with one of the readily available plotting functionalities. Most importantly, the crosslinking data can be exported to the required data format of more than ten different down-stream analysis tools, eliminating the need for manual data re-shaping and transformation. pyXLMS is available as a python package and as a web application with graphical user interface.

Table 1 | Supported crosslink search engines, tools, and formats in pyXLMS including indicators of whether the specific search engines/tools/formats support crosslink-spectrum-matches (CSMs) and/or crosslinks/peptide pairs/residue pairs (XLs)

Search Engine/Tool/Format	CSMs	XLs	Supported File Formats
MaxQuant/MaxLynx[15]	yes	no	txt
MeroX[35,36]	yes	no	csv, zhrm
MS Annika[13,37,38]	yes	yes	pdResult, xlsx, txt
pLink[39]	yes	yes	csv
Scout[14]	yes	yes	csv
xiSearch[40] & xiFDR[41]	yes	yes	csv
xiNET[42] & xiVIEW[26]	yes	yes	csv
XlinkX[43]	yes	yes	pdResult, xlsx, txt
mzIdentML[31–33]	yes	no	mzid
pyXLMS/custom formats	yes	yes	parquet, xlsx, csv

pyXLMS also allows to easily create intersections of CSMs and XLs, either from different replicates or different crosslink search engines, which has been shown to increase specificity and lower the number of false positive identifications[51] but does reduce sensitivity. Visualization of intersections and generally overlaps of different sets of CSMs and XLs is also possible via Venn diagram plotting that is available in pyXLMS.

Last but not least, the most important feature in pyXLMS is the ability to export results to the formats needed by popular crosslink down-stream analysis tools. pyXLMS currently already supports more than ten different down-stream analysis tools and formats and handles all data re-shaping and transformation steps that are necessary. pyXLMS also implements the ProForma notation of the HUPO Proteomics Standards Initiative[53] and can convert CSMs and XLs to ProForma. A complete list of supported down-stream analysis tools is given in Table 2.

An in-depth description of the implementation of the python package is given in Section "Implementation of pyXLMS".

pyXLMS simplifies crosslink analysis independently of the used crosslink search engine

In order to demonstrate the applicability of pyXLMS we have re-analyzed a previously published crosslink dataset with three different crosslink search engines and show how pyXLMS unifies the down-stream analysis workflow with no need for adoption to the search engine specific result formats.

We retrieved data from PRIDE[54] where purified recombinant *S. pyogenes* Cas9 fused with a Halo-tag was crosslinked with DSBSO[55] in three replicates[11]. Subsequently, we searched the data with analogous settings using the crosslink search engines pLink[39], Scout[14], and XlinkX[43]. With pyXLMS we were able to directly load the result files from all searches, get an overview of the identified CSMs and XLs as shown in Fig. 2, and immediately continue with down-stream analysis via the export to xiVIEW[26], XMAS[50], and AlphaLink2[23] as depicted in Fig. 3. Crosslinks were grouped by their protein crosslink positions, therefore the term XLs refers to unique residue pairs in this section.

For this dataset pLink outperformed the other search engines not only in terms of number of identified CSMs and XLs at 1% estimated FDR with an average of 765 CSMs and 428 XLs per replicate, but also in terms of overlap between replicates with 293 XLs that are identified across all three replicates. Scout reported on average 285 CSMs and 216 XLs at an estimated FDR of 1% with an overlap of 94 XLs that were

detected in all three replicates. The crosslink search engine XlinkX identified on average 172 CSMs and 146 XLs per replicate at 1% estimated FDR with 43 XLs that were detected in all three replicates. Generally the agreement between replicates is best for pLink and gradually gets more disjoint for both Scout and XlinkX. We investigated the intersections for all crosslink search engines with xiVIEW and ultimately took an intersection of all XLs that were found in all replicates by all search engines which resulted in a total of 27 XLs (see Fig. 2B) that we used for further structural investigations with XMAS and AlphaLink2. Taking the intersection for down-stream analysis reduces the chance of accidentally including a false-positive crosslink identification[51].

Table 2 | Supported export formats and down-stream analysis tools in pyXLMS including indicators whether crosslink-spectrum-matches (CSMs) and/or crosslinks/peptide pairs/residue pairs (XLs) can be exported

Tool/Format	CSMs	XLs
AlphaLink2[23]	no	yes
IMP-X-FDR[51]	yes	yes
ProXL[24,25]	yes	no
PyXlinkViewer (PyMOL)[49]	no	yes
xiFDR[41]	yes	no
xiNET[42]	no	yes
xiVIEW[26]	yes	yes
XlinkDB[44–47]	no	yes
xlms-tools[48]	no	yes
XMAS (ChimeraX)[50]	no	yes

We want to emphasize here that these results are not a benchmark of the individual crosslink search engines which all use different FDR estimation methods that we did not investigate since our goal is to highlight the capabilities of pyXLMS. Plenty of publications already exist that compare and benchmark different crosslink search engines[13,14,51,56]. It should also be noted here that for our workflow we used the Scout version of The Liu Lab but another Scout version that produces different results also exists (see also section "Data analysis").

Figure 3 shows the results of the down-stream analysis. We visualized unambiguous intra-crosslinks of Cas9 that were identified in all three replicates at 1% estimated FDR via xiVIEW[26] for the crosslink search engines pLink[39] (Fig. 3A), Scout[14] (Fig. 3B), and XlinkX[43] (Fig. 3C), again also confirming that pLink identifies the most Cas9 intra-crosslinks. We then took the intersection of all these crosslinks with pyXLMS (27 crosslinks in total after intersection) and used them to investigate the structure of the crosslinked Cas9. In order to facilitate that, we predicted the structure of Cas9 once with AlphaFold2[57], and once with AlphaLink2[23] directly using the files generated with pyXLMS. In comparison to AlphaFold2, AlphaLink2 additionally uses the crosslink constraints to predict the protein structure. Subsequently, we visualized the predicted structures and crosslinks with XMAS[50] in ChimeraX[28–30] as shown in Fig. 3E, F. We examined how many of the crosslinks satisfied or violated the crosslinker specific distance

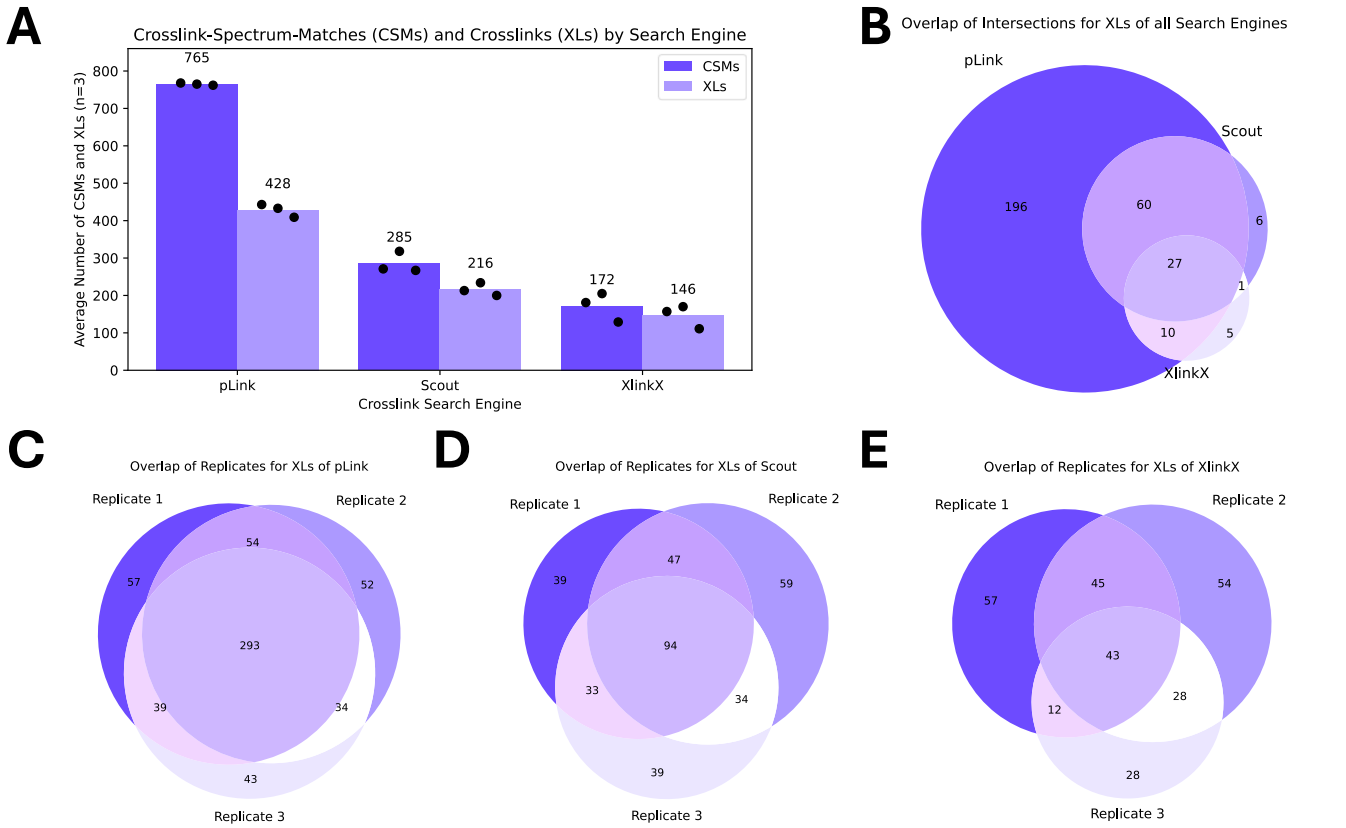


Fig. 2 | Overview of the results from pLink[39], Scout[14], and XlinkX[43] on the Cas9 crosslinking data[11] created with pyXLMS. A On average pLink identifies 765 CSMs and 428 XLs (unique residue pairs) per replicate, while Scout identifies 285 CSMs and 216 XLs on average per replicate. XlinkX reports an average of 172 identified CSMs and 146 identified XLs per replicate. All results are validated for 1% estimated FDR. Individual data points for the three replicates ($n = 3$) are shown as dots. **B** Overlap of identified XLs for pLink, Scout, and XlinkX when only taking XLs that are found across all three replicates by each search engine. Essentially this is a Venn diagram of the intersections of **C**, **D**, and **E**. **C** Overlap of the XLs identified by pLink across all three replicates. 293 XLs are identified in all three replicates. **D** Overlap of the XLs identified by Scout across all three replicates with 94 XLs which are identified in all three replicates. **E** Overlap of the XLs identified by XlinkX across all three replicates where 43 XLs are identified in all replicates.

https://doi.org/10.1038/s41467-026-77407-1

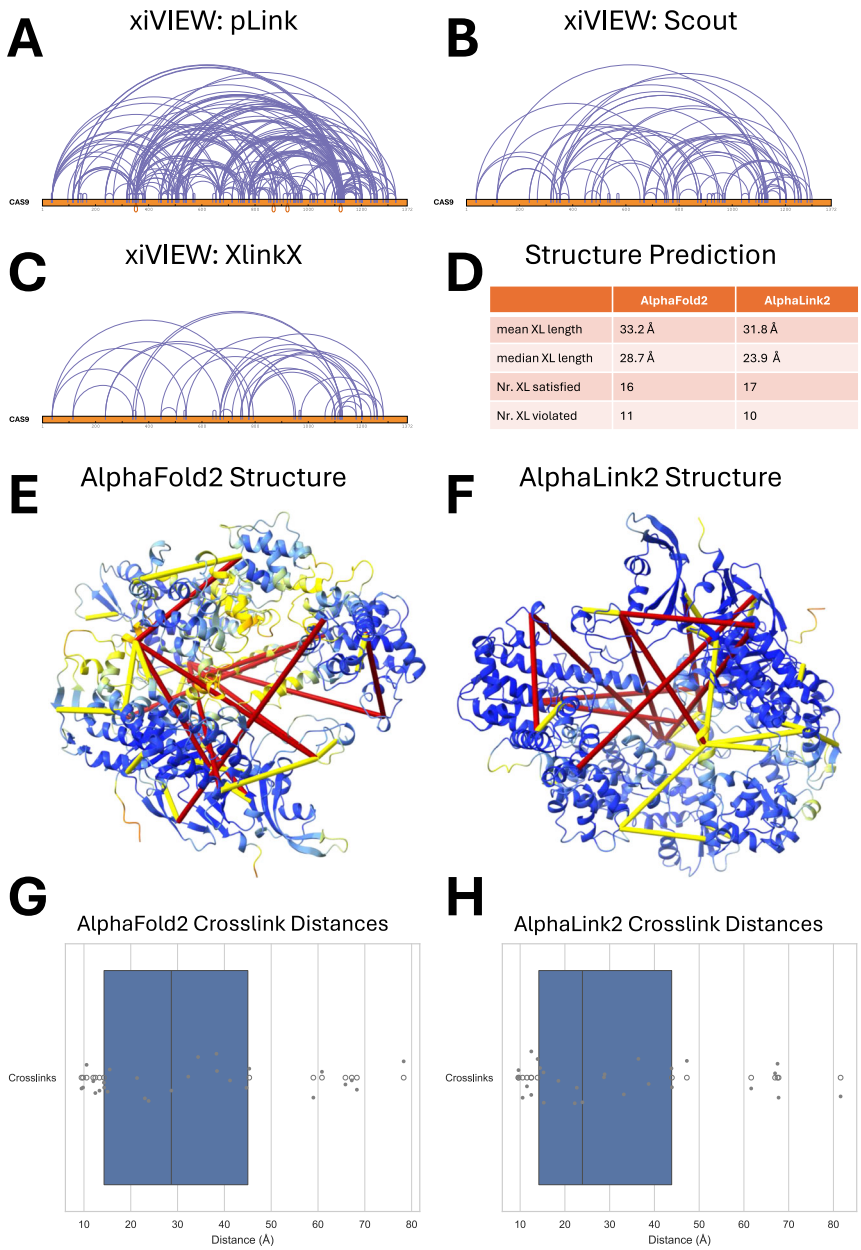


Fig. 3 | Overview of the down-stream analysis of the crosslink results. Panels **A-C** show all unambiguous Cas9 intra-crosslinks mapped to the sequence of Cas9 with xiVIEW[26] for crosslinks that were validated at 1% estimated FDR and identified in all three replicates by the search engine (**A**) pLink[39], (**B**) Scout[14], and (**C**) XlinkX[43]. We then took the intersection of A-C and mapped it to the AlphaFold2[57] predicted structure of Cas9 and the structure predicted by AlphaLink2[23] which additionally uses the crosslink constraints. Panel (**D**) shows a comparsion of the two predicted structures in terms of agreement with the crosslink results with the AlphaLink2 structure generally showing small improvements over the AlphaFold2 predicted structure. Panel (**E**) shows the AlphaFold2 predicted structure and panel (**F**) the AlphaLink2 predicted structure visualized by XMAS[50] with crosslinks that satisfy the distance threshold in yellow (16 for AlphaFold2, 17 for AlphaLink2) and crosslinks that violate the distance threshold in red (11 for AlphaFold2, 10 for AlphaLink2). The structures are colored by pLDDT using the standard AlphaFold coloring[57]. Panels **G** and **H** show the crosslink distance distributions for the (**G**) AlphaFold2 predicted structure and (**H**) the AlphaLink2 predicted structure. The AlphaLink2 predicted structure has slightly lower mean and median crosslink distances.

constraint of DSBSO[55] which we chose as 35Å as suggested by Jiao et al.[58]. In total 16 crosslinks satisfied the distance constraint in the AlphaFold2 predicted structure and 17 crosslinks in the AlphaLink2 predicted structure, showing a slight improvement when using AlphaLink2. In turn, there were 11 crosslinks that violated the distance constraint in the AlphaFold2 predicted structure and 10 crosslinks in the AlphaLink2 predicted structure. Overall the mean and median crosslink distance decreased from 33.2Å to 31.8Å and from 28.7Å to 23.9Å respectively, when using AlphaLink2 compared to AlphaFold2 as shown in the crosslink distance distribution plots in Fig. 3G, H. The model quality is good for both structures with an average pLDDT of 86.8 and 93.7 for AlphaFold2 and AlphaLink2 correspondingly, and the many observed over-length crosslinks are explained by the flexibility of Cas9 that inevitably cannot be easily captured in a static snapshot.

The pyXLMS web application provides an easy to use interface for non-programmers

We have built a web application on top of the pyXLMS python package for researchers without a bioinformatics background that offers an intuitive graphical user interface and almost all the functionality of the underlying python package without the need for any programming experience. The web application is divided into four different tabs that

https://doi.org/10.1038/s41467-026-77407-1

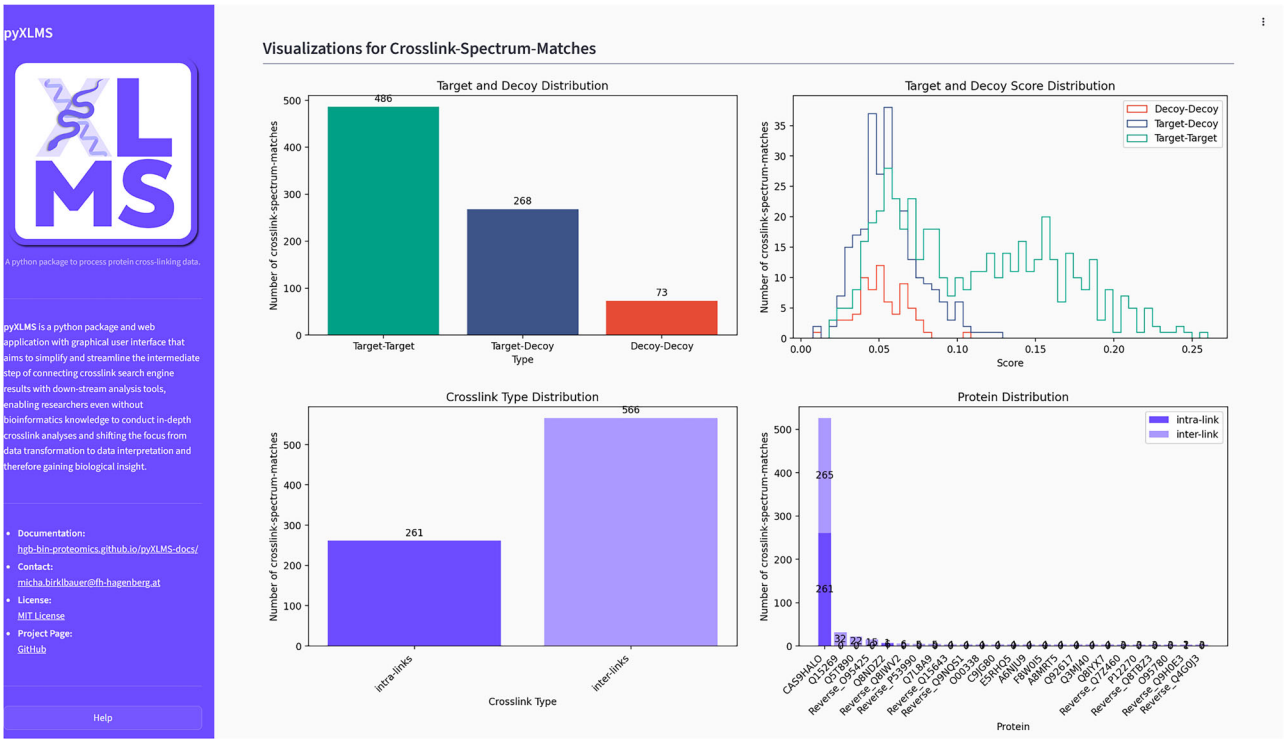


Fig. 4 | The pyXLMS web application available via https://hgb-bin-proteomics.github.io/pyXLMS-app offers a programming-free experience with an intuitive graphical user interface to transform and export crosslinking data. The here displayed visualization tab shows different plots describing the uploaded cross-linking data. In the top left an overview of the identified target-target, target-decoy, and decoy-decoy matches is shown. The top right displays a score distribution for all CSMs. The bottom left shows the distribution of identified intra- and inter-crosslinks. The bottom right shows the most frequently crosslinked proteins by both intra- and inter-crosslinks. The figure shows data of CSMs identified with Scout[14] in replicate one of the previously used dataset[11], see Section "pyXLMS simplifies crosslink analysis independently of the used crosslink search engine".

inherently guide researchers through the app. First, researchers can upload their results, choose a crosslink search engine or format, select a crosslinker and are then able to import their data. Secondly, users may filter their data for their desired CSMs and XLs. Thirdly, the visualization tab—as depicted in Fig. 4—shows various different descriptive and qualitative plots of the filtered data and users can confirm that they selected CSMs and XLs that they are interested in. The visualization tab also allows input of a STRING[52] organism which will then display the STRING score distributions for all inter-links (shown in Supplementary Fig. 2). Last but not least, researchers can export the data to one or more of the supported down-stream analysis tools to further investigate their data for biological significance. Importantly, the export options are largely dependent on the uploaded data, e.g. uploading CSMs will only show export options that support CSMs and uploading XLs will only show export options that support XLs (refer to Table 2). In any case CSMs can always be aggregated to XLs within the web application to show all export options.

Some functionality of the python package is not available in the web application, for example creating intersections of results from different crosslink search engines as performed for the analysis shown in "pyXLMS simplifies crosslink analysis independently of the used cross-link search engine". Supplementary Section 2 lists all features that are not available in the web application and includes links to step-by-step instructions on how to perform these actions with the python package.

We are hosting a free and publicly available instance of the pyXLMS web application at https://hgb-bin-proteomics.github.io/pyXLMS-app, however we also support self-hosting or running the web application locally - it is cross-platform and runs on any computer that supports python and a web browser. Instructions for self-hosting, running it locally, and a user guide for navigating trough the app are available via https://hgb-bin-proteomics.github.io/pyXLMS-docs or via the Supplementary Information.

pyXLMS is well documented with step-by-step instructions for both programmers and non-programmers

One core focus of pyXLMS is usability and to empower all researchers in crosslinking, whether they are in the wet or dry lab, newcomers or experts in the field, or even developers of their own tools. We have compiled extensive resources that document all functionality in pyXLMS which includes not only a package reference that documents all of the python module but also an extensive user guide featuring many examples of common usage patterns—for both the python package and the web application. The user guide is available via https://hgb-bin-proteomics.github.io/pyXLMS-docs and consists of an in-depth explanation of all functionality, important background information about the crosslink search engines and down-stream analysis tools, and many examples that include step-by-step instructions. We have implemented an assortment of Jupyter notebooks to follow along, with different example data to get a hands-on experience with both pyXLMS and the supported down-stream analysis tools. The user guide is written for researchers with and without programming experience and includes instructions well beyond pyXLMS including the installation of all required tools. An exemplary excerpt of the user guide is shown in Supplementary Fig. 3.

For developers we have created a complete python package reference at https://hgb-bin-proteomics.github.io/pyXLMS/ and have setup contribution guidelines in the pyXLMS repository.

Discussion

We here present pyXLMS, a python package and web application with graphical user interface that aims to simplify and streamline the

intermediate step of connecting crosslink search engine results with down-stream analysis tools which has remained a manual, cumbersome, and error prone step in the data analysis that often requires extensive data re-shaping or transformation. This step is only possible for scientists with a strong bioinformatics background and yet is crucial for gaining biological insight from the identified crosslinks. pyXLMS is enabling scientists even without bioinformatics knowledge to conduct in-depth crosslink analyses and shifting the focus from data transformation to data interpretation. pyXLMS currently already supports input from several different crosslink search engines and tools, including MaxLynx[15], MeroX[35,36], MS Annika[13,37,38], pLink 2 and pLink 3[39], Scout[14], xiSearch[40] and the validation tool xiFDR[41], xiNET[42], xiVIEW[26], XlinkX[43], as well as the mzIdentML format[31–33] of the HUPO Proteomics Standards Initiative, and a well-documented and human-readable custom tabular format. Data processing and quality control is facilitated by functionality that is directly available within pyXLMS such as aggregation, validation, annotation, filtering, and visualization of CSMs and XLs. Most importantly, pyXLMS enables easy export to the required data format of more than ten different down-stream analysis tools such as AlphaLink2[23], ProXL[24,25], xiNET[42], xiVIEW[26], xiFDR[41], XlinkDB[44–47], xlms-tools[48], PyMOL[27](via pyXlinkViewer[49]), ChimeraX[28–30](via XMAS[50]), or IMP-X-FDR[51].

The introduction of the mzIdentML format[31] developed by the HUPO Proteomics Standards Initiative aimed to address the interoperability issue within the crosslinking community already in 2017 with dedicated support for crosslink data in version 1.2[32] which was further extended specifically for the reporting of crosslink results recently in version 1.3[33]. Unfortunately, many crosslink search engines and down-stream analysis tools have yet to adopt the format—of the nine currently supported crosslink search engines and input tools in pyXLMS only four support mzIdentML export (MS Annika, Scout, xiFDR, and XlinkX), and only four of the 10 supported down-stream analysis tools import mzIdentML (AlphaLink2, xiFDR, xiVIEW, and XMAS). This means that of the currently supported I/O landscape in pyXLMS only about 18% (16/90) are covered by mzIdentML, all other combinations of input and output tools would need to be manually implemented with custom data transformation scripts if pyXLMS did not exist. Beyond its data processing and quality control capabilities, we propose pyXLMS not as a replacement or alternative for mzIdentML—mzIdentML is still the recommended way of reporting crosslink results, especially at the CSM level—but as a much needed foundational layer for interacting with crosslinking data and as an extension for all cases where mzIdentML is either not available, not supported, or not preferred. Ultimately pyXLMS empowers researchers, giving them the choice to use whatever files and formats they have and feel confident with.

We demonstrated the applicability and benefits of pyXLMS by re-analyzing a publicly available crosslink dataset[11] of the protein Cas9 crosslinked with DSBSO[55]. We show how the same data analysis workflow including down-stream analysis tools xiVIEW[26], XMAS[50], and AlphaLink2[23] can be applied across the different crosslink search engines pLink[39], Scout[14], and XlinkX[43] using pyXLMS. The integration of pyXLMS allows us to switch between search engines and analysis tool at will, eliminating the need for manual adoption to the various data formats.

Despite our best efforts pyXLMS still comes with some limitations that are a direct result of the differences in the output formats of crosslink search engines. Many crosslink search engines do not report any kind of decoy matches which makes validation in pyXLMS or export to xiFDR[41] impossible which is why we recommend using validated results for pyXLMS. Validation within pyXLMS is currently only supported for MaxLynx[15], MS Annika[13], and xiSearch[40]. Furthermore, the different down-stream analysis tools require varying input information which might not be consistently available from all crosslink search engines. Some of this can be mitigated by functionality in pyXLMS such as annotation or by additional information that needs to be passed to pyXLMS for a successful export—which users are guided through in both the python package and web application. Generally, the export to all down-stream analysis tools should work for all crosslink search engines and input formats, with the exception of the export to xiFDR which is limited to MaxLynx, MS Annika, and xiSearch for above reasons. For safety pyXLMS makes sure before the export that all the required information is available and will otherwise throw an error.

Additionally, pyXLMS focuses on reading, processing, and exporting data on CSM and XL aggregation levels and does not support PPI level data, neither in reading nor processing. This has been an intentional design choice as the number of crosslink search engines supporting PPI level results is still limited and currently none of the down-stream analysis tools integrated with pyXLMS require PPI input. We recommend users who are particularly interested in PPI-level analysis to preferably use a crosslink search engine that does support PPI level results (e.g. Scout[14]) or that supports validation via xiFDR[41] (e.g. xiSearch[40], MS Annika[13]) which can handle aggregation and validation up to the PPI level. Results at the PPI level can then be easily further analyzed in tools like Cytoscape[59]. Generally, pyXLMS can transform lower-level results for usage with xiFDR to support this process, or alternatively export them to xiVIEW[26] which can also aggregate CSMs and XLs to PPI networks.

pyXLMS is publicly available under a permissive open-source license via https://github.com/hgb-bin-proteomics/pyXLMS and we are not only welcoming but hoping for contributions by the community with the aim of supporting as many crosslink search engines and down-stream analysis tools as possible. Moreover, as long time contributors to the field of XLMS we are committed to maintain the project and drive development forward. We think that pyXLMS is an important step in making XLMS data analysis more accessible to everyone and believe that it has the potential to become a foundational building block in future crosslinking tools.

Methods

Implementation of pyXLMS

pyXLMS is implemented in python using a functional programming approach and divided into currently seven submodules: (1) *parser* handles reading of all input files from the different crosslink search engines and data formats. For reading of tabular data the python package *pandas*[60,61] is used, for reading of mzIdentML files we make use of *pyteomics*[62,63]. All data is parsed into python native data structures for performance and serializability, this is further described in the pyXLMS documentation. Starting with pyXLMS version 2.0.0 we now use *Pydantic BaseModel* https://pydantic.dev/docs/validation/ classes instead for increased type safety and data validation, while preserving performance and serializability. These core data structures are implemented in the (2) *data* submodule. (3) The submodule *transform* handles all data transformation steps including filtering, annotation, and validation. For reading FASTA files we use *biopython*[64], all other functionality is natively implemented. For integration with STRING we use the public STRING API[52]. For validation we implemented the FDR estimation algorithm as used by MS Annika[13] as well as the formula suggested by Fischer and Rappsilber for directional crosslinks[41]. (4) The *plotting* submodule implements all visualization functionality and mainly uses the python package *matplotlib*[65]. (5) The *exporter* submodule handles exporting CSMs and XLs to the formats required by the supported down-stream analysis tools. The python package *biopandas*[66] is used for reading PDB files and *biopython*[64] is used for sequence alignment. We use *lxml* for XML schema validation of ProXL[24,25] exports. (6) In the *pipelines* submodule we provide a standard data processing pipeline that performs deduplication, grouping, validation, and target-target filtering which is the most commonly used input for down-stream analysis tools. (7) The last

submodule called *constants* contains frequently used data such as crosslinker names and their masses, modifications, amino acid codes, and other biological and technical properties used throughout pyXLMS.

The web application is implemented in *streamlit*. For documentation we use *sphinx* and *nextra*. All python code is linted with *ruff*, type checked with *pyright* and *ty*, and tested with *pytest*. We currently have over 650 tests covering the codebase of pyXLMS (over 85% test coverage), that includes tests for parsing of the result files of all supported crosslink search engines. We tested result files from MaxLynx as part of MaxQuant version 2.6.2.0[15,34], MeroX version 2.0.1.4[35,36], MS Annika version 3.0.7[13], pLink version 2.3.11 and version 3.0.17[39], Scout versions 1.6.3 and 2.1.0 (released by Diogo Borges, https://github.com/diogobor/Scout/) and version 2.0.0 beta (released by The Liu Lab, https://github.com/theliulab/Scout/)[14], xiSearch version 1.7.6.7[40], xiFDR version 2.2.1[41], XlinkX as part of Thermo Fisher Scientific's Proteome Discoverer versions 3.1.0.638 and 3.2.0.450[43], as well as the mzIdentML format version 1.2 and version 1.3[32,33].

Data acquisition

All used mass spectrometry proteomics data was retrieved from the ProteomeXchange consortium (http://proteomecentral.proteomexchange.org)[67] via the PRIDE partner repository[54] with the dataset identifier PXD061173[11]. In short, the authors prepared purified recombinant Cas9 from *S. pyogenes* fused with a Halo-tag and crosslinked it with DSBSO. The crosslinked Cas9 samples were spiked in a 1:80 (w/w, 20 μg Cas9 + 1.6 mg HeLa) ratio into a HeLa protein digest, followed by DBCO bead-based enrichment. Samples were then analyzed using a Vanquish Neo UHPLC system and an Orbitrap Eclipse Tribrid mass spectrometer (both Thermo Fisher Scientific). A detailed description of all sample preparation, enrichment, and data acquisition steps is given in the authors' original publication[11].

Data analysis

RAW files were directly searched with the crosslink search engines pLink[39] (version 3.0.17), Scout[14] (version 2.0.0 beta, retrieved from The Liu Lab https://github.com/theliulab/Scout/; and version 2.1.0, retrieved from the Borges repository https://github.com/diogobor/Scout/), and XlinkX[43] (version as part of Thermo Fisher Scientific's Proteome Discoverer version 3.2.0.450) using the full human SwissProt reference proteome (n = 20 328, UniProt Proteome ID UP000005640, retrieved 03. November 2025) and the sequence of Cas9 as the protein database. Search settings were set analogous for all search engines using digestion enzyme trypsin with a maximum of three missed cleavages allowed, a minimum peptide length of six amino acids and a maximum peptide length of 30 amino acids. Carbamidomethylation of cysteine was set as a fixed modification and oxidation of methionine was set as a variable modification for all peptides. For non-crosslinked peptides we also considered the amidated, hydrolyzed and tris forms of DSBSO[55] as variable modifications for all lysine residues. The crosslinker parameter was set to DSBSO with reactions to lysine and the protein n-terminus allowed. The precursor mass tolerance was set to 5 ppm and the fragment mass tolerance to 10 ppm. For XlinkX the "Cleavable_MS2" workflow was used. Results were validated for 1% estimated FDR using the in-built FDR estimation algorithms of all crosslink search engines and setting the target FDR to 1% at all possible levels. All other crosslink search engine parameters were left to their defaults.

For Scout only the results from The Liu Lab version were used for further down-stream processing and ultimately shown in this manuscript as that version achieved higher numbers of identified CSMs and XLs in all cases. A comparison of the two Scout versions can be found in the manuscript's GitHub repository as described in Section "Data availability".

The validated crosslink search engine results were then directly loaded with pyXLMS (version 1.7.3). Subsequently, re-annotation of associated proteins and protein crosslink positions using the *transform.reannotate_positions()* function of pyXLMS was performed to have consistent annotation accross all search engine results. Any non-unique CSMs and crosslinks were filtered out using the pyXLMS function *transform.unique()* via grouping by protein crosslink position (= unique residue pairs). Crosslink intersections were created using the pyXLMS intersection and Venn diagram functionalities, and were finally exported to xiVIEW[26], XMAS[50], and AlphaLink2[23] format using the pyXLMS exporter submodule.

Results were then visualized with xiVIEW, modelled with AlphaLink2 (version git@d54bcbf), and analyzed with XMAS (version 1.1.3) in ChimeraX[28–30] (version 1.10.1)—all directly with the outputs created by pyXLMS. For the comparison to AlphaFold2 we predicted the structure of Cas9 with ColabFold[68] using the AlphaFold2[57] notebook with MMseqs2 support[69] (notebook version 1.5.5). The sequence of Cas9 was used as input and all other ColabFold parameters were left at their default values. The rank 1 structure was used for comparison with the AlphaLink2 predicted structure.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

All used mass spectrometry proteomics data is available from the ProteomeXchange consortium (http://proteomecentral.proteomexchange.org)[67] via the PRIDE partner repository[54] with the dataset identifier PXD061173[11]. Result files of the re-analysis, structure prediction, scripts and figure code used for this manuscript are available via the GitHub repository https://github.com/hgb-bin-proteomics/pyXLMS-manuscript.

Code availability

pyXLMS requires at least python version 3.7 and runs on any operating system or hardware. The python package is available from the Python Package Index (PyPI) via https://pypi.org/project/pyXLMS/ and can easily be installed via "pip install pyxlms". The source code for pyXLMS is available via the GitHub repository https://github.com/hgb-bin-proteomics/pyXLMS under an MIT license. The web application is hosted by the University of Applied Sciences Upper Austria and accessible free of charge via https://hgb-bin-proteomics.github.io/pyXLMS-app. Source code for the web application is part of the pyXLMS git repository. Documentation for the python package is available at https://hgb-bin-proteomics.github.io/pyXLMS/. A comprehensive user guide and documentation for both the python package and the web application is available at https://hgb-bin-proteomics.github.io/pyXLMS-docs and https://pyxlms.dev. An overview of all published pyXLMS resources is also given in the Supplementary Information in Supplementary Section 1.

References

1. O'Reilly, F. J. & Rappsilber, J. Cross-linking mass spectrometry: methods and applications in structural, molecular and systems biology. *Nature Structural & Molecular Biology* **25**, 1000–1008, https://doi.org/10.1038/s41594-018-0147-0 (2018).
2. Petrotchenko, E. V. & Borchers, C. H. Crosslinking combined with mass spectrometry for structural proteomics. *Mass Spectrometry Reviews* **29**, 862–876, https://doi.org/10.1002/mas.20293 (2010).
3. Shi, Y. et al. Structural characterization by cross-linking reveals the detailed architecture of a coatomer-related heptameric module from the nuclear pore complex. *Molecular & Cellular Proteomics* **13**, 2927–2943, https://doi.org/10.1074/mcp.m114.041673 (2014).
4. Weisbrod, C. R. et al. In vivo protein interaction network identified with a novel real-time cross-linked peptide identification strategy.

Journal of Proteome Research **12**, 1569–1579, https://doi.org/10.1021/pr3011638 (2013).
5. Ser, Z., Cifani, P. & Kentsis, A. Optimized cross-linking mass spectrometry for in situ interaction proteomics. *Journal of Proteome Research* **18**, 2545–2558, https://doi.org/10.1021/acs.jproteome.9b00085 (2019).
6. Kao, A. et al. Development of a novel cross-linking strategy for fast and accurate identification of cross-linked peptides of protein complexes. *Molecular & Cellular Proteomics* **10**, M110.002170, https://doi.org/10.1074/mcp.m110.002212 (2011).
7. Steigenberger, B., Pieters, R. J., Heck, A. J. R. & Scheltema, R. A. Phox: An imac-enrichable cross-linking reagent. *ACS Central Science* **5**, 1514–1522, https://doi.org/10.1021/acscentsci.9b00416 (2019).
8. Müller, F. et al. Developing a new cleavable crosslinker reagent for in-cell crosslinking. *Commun. Chem.* **8**, https://doi.org/10.1038/s42004-025-01568-1 (2025).
9. Schnirch, L. et al. Expanding the depth and sensitivity of cross-link identification by differential ion mobility using high-field asymmetric waveform ion mobility spectrometry. *Analytical Chemistry* **92**, 10495–10503, https://doi.org/10.1021/acs.analchem.0c01273 (2020).
10. Wheat, A. et al. Protein interaction landscapes revealed by advanced in vivo cross-linking–mass spectrometry. *Proc. Natl Acad. Sci.***118**, https://doi.org/10.1073/pnas.2023360118 (2021).
11. Bräuer, P., Tirian, L., Müller, F., Mechtler, K. & Matzinger, M. In vivo crosslinking and effective 2d enrichment for proteome wide interactome studies. *Commun. Chem.* **8**, https://doi.org/10.1038/s42004-025-01644-6 (2025).
12. Müller, F. et al. Breaking barriers in crosslinking mass spectrometry with enhanced throughput and sensitivity using orbitrap astral. *Nat. Commun.***16**, https://doi.org/10.1038/s41467-025-64844-7 (2025).
13. Birklbauer, M. J. et al. Proteome-wide non-cleavable crosslink identification with ms annika 3.0 reveals the structure of the c. elegans box c/d complex. *Commun. Chem.* **7**, https://doi.org/10.1038/s42004-024-01386-x (2024).
14. Clasen, M. A. et al. Proteome-scale recombinant standards and a robust high-speed search engine to advance cross-linking ms-based interactomics. *Nature Methods*https://doi.org/10.1038/s41592-024-02478-1 (2024).
15. Yılmaz, Ş, Busch, F., Nagaraj, N. & Cox, J. Accurate and automated high-coverage identification of chemically cross-linked peptides with maxlynx. *Analytical Chemistry* **94**, 1608–1617, https://doi.org/10.1021/acs.analchem.1c03688 (2022).
16. Leitner, A. et al. Probing native protein structures by chemical cross-linking, mass spectrometry, and bioinformatics. *Molecular & Cellular Proteomics* **9**, 1634–1649, https://doi.org/10.1074/mcp.r000001-mcp201 (2010).
17. Iacobucci, C., Götze, M. & Sinz, A. Cross-linking/mass spectrometry to get a closer view on protein interaction networks. *Current Opinion in Biotechnology* **63**, 48–53, https://doi.org/10.1016/j.copbio.2019.12.009 (2020).
18. Piersimoni, L. & Sinz, A. Cross-linking/mass spectrometry at the crossroads. *Analytical and Bioanalytical Chemistry* **412**, 5981–5987, https://doi.org/10.1007/s00216-020-02700-x (2020).
19. Matzinger, M. & Mechtler, K. Cleavable cross-linkers and mass spectrometry for the ultimate task of profiling protein–protein interaction networks in vivo. *Journal of Proteome Research* **20**, 78–93, https://doi.org/10.1021/acs.jproteome.0c00583 (2020).
20. Richards, A. L., Eckhardt, M. & Krogan, N. J. Mass spectrometry-based protein–protein interaction networks for the study of human diseases. *Mol. Syst. Biol.* **17**, https://doi.org/10.15252/msb.20188792 (2021).
21. Belsom, A. & Rappsilber, J. Anatomy of a crosslinker. *Current Opinion in Chemical Biology* **60**, 39–46, https://doi.org/10.1016/j.cbpa.2020.07.008 (2021).
22. Faustino, A. M., Sharma, P., Manriquez-Sandoval, E., Yadav, D. & Fried, S. D. Progress toward proteome-wide photo-cross-linking to enable residue-level visualization of protein structures and networks in vivo. *Analytical Chemistry* **95**, 10670–10685, https://doi.org/10.1021/acs.analchem.3c01369 (2023).
23. Stahl, K. et al. Modelling protein complexes with crosslinking mass spectrometry and deep learning. *Nat. Commun.* **15**, https://doi.org/10.1038/s41467-024-51771-2 (2024).
24. Riffle, M., Jaschob, D., Zelter, A. & Davis, T. N. Proxl (protein cross-linking database): A platform for analysis, visualization, and sharing of protein cross-linking mass spectrometry data. *Journal of Proteome Research* **15**, 2863–2870, https://doi.org/10.1021/acs.jproteome.6b00274 (2016).
25. Riffle, M., Jaschob, D., Zelter, A. & Davis, T. N. Proxl (protein cross-linking database): A public server, qc tools, and other major updates. *Journal of Proteome Research* **18**, 759–764, https://doi.org/10.1021/acs.jproteome.8b00726 (2018).
26. Combe, C. W., Graham, M., Kolbowski, L., Fischer, L. & Rappsilber, J. xiview: Visualisation of crosslinking mass spectrometry data. *J. Mol. Biol.* 168656, https://doi.org/10.1016/j.jmb.2024.168656 (2024).
27. Schrödinger, LLC. The PyMOL molecular graphics system (2015).
28. Goddard, T. D. et al. Ucsf chimerax: Meeting modern challenges in visualization and analysis. *Protein Science* **27**, 14–25, https://doi.org/10.1002/pro.3235 (2017).
29. Pettersen, E. F. et al. Ucsf chimerax: Structure visualization for researchers, educators, and developers. *Protein Science* **30**, 70–82, https://doi.org/10.1002/pro.3943 (2020).
30. Meng, E. C. et al. Ucsf chimerax: Tools for structure building and analysis. *Protein Sci.* **32**, https://doi.org/10.1002/pro.4792 (2023).
31. Jones, A. R. et al. The mzidentml data standard for mass spectrometry-based proteomics results. *Molecular & Cellular Proteomics* **11**, M111.014381–1–M111.014381–10, https://doi.org/10.1074/mcp.m111.014381 (2012).
32. Vizcaíno, J. A. et al. The mzidentml data standard version 1.2, supporting advances in proteome informatics. *Molecular & Cellular Proteomics* **16**, 1275–1285, https://doi.org/10.1074/mcp.m117.068429 (2017).
33. Combe, C. W. et al. mzidentml 1.3.0 – essential progress on the support of crosslinking and other identifications based on multiple spectra. *PROTEOMICS* **24**, https://doi.org/10.1002/pmic.202300385 (2024).
34. Cox, J. & Mann, M. Maxquant enables high peptide identification rates, individualized p.p.b.-range mass accuracies and proteome-wide protein quantification. *Nature Biotechnology* **26**, 1367–1372, https://doi.org/10.1038/nbt.1511 (2008).
35. Götze, M. et al. Stavrox—a software for analyzing crosslinked products in protein interaction studies. *Journal of the American Society for Mass Spectrometry* **23**, 76–87, https://doi.org/10.1007/s13361-011-0261-2 (2011).
36. Iacobucci, C. et al. A cross-linking/mass spectrometry workflow based on ms-cleavable cross-linkers and the merox software for studying protein structures and protein–protein interactions. *Nature Protocols* **13**, 2864–2889, https://doi.org/10.1038/s41596-018-0068-8 (2018).
37. Pirklbauer, G. J. et al. Ms annika: A new cross-linking search engine. *Journal of Proteome Research* **20**, 2560–2569, https://doi.org/10.1021/acs.jproteome.0c01000 (2021).
38. Birklbauer, M. J., Matzinger, M., Müller, F., Mechtler, K. & Dorfer, V. Ms annika 2.0 identifies cross-linked peptides in ms2–ms3-based workflows at high sensitivity and specificity. *Journal of Proteome*

Research **22**, 3009–3021, https://doi.org/10.1021/acs.jproteome.3c00325 (2023).
39. Chen, Z.-L. et al. A high-speed search engine plink 2 with systematic evaluation for proteome-scale identification of cross-linked peptides. *Nat. Commun.* **10**, https://doi.org/10.1038/s41467-019-11337-z (2019).
40. Mendes, M. L. et al. An integrated workflow for crosslinking mass spectrometry. *Mol. Syst. Biol.* **15**, https://doi.org/10.15252/msb.20198994 (2019).
41. Fischer, L. & Rappsilber, J. Quirks of error estimation in cross-linking/mass spectrometry. *Analytical Chemistry* **89**, 3829–3833, https://doi.org/10.1021/acs.analchem.6b03745 (2017).
42. Combe, C. W., Fischer, L. & Rappsilber, J. xinet: Cross-link network maps with residue resolution. *Molecular & Cellular Proteomics* **14**, 1137–1147, https://doi.org/10.1074/mcp.o114.042259 (2015).
43. Liu, F., Lössl, P., Scheltema, R., Viner, R. & Heck, A. J. R. Optimized fragmentation schemes and data analysis strategies for proteome-wide cross-link identification. *Nat. Commun.* **8**, https://doi.org/10.1038/ncomms15473 (2017).
44. Zheng, C. et al. Xlink-db: Database and software tools for storing and visualizing protein interaction topology data. *Journal of Proteome Research* **12**, 1989–1995, https://doi.org/10.1021/pr301162j (2013).
45. Schweppe, D. K. et al. Xlinkdb 2.0: integrated, large-scale structural analysis of protein crosslinking data. *Bioinformatics* **32**, 2716–2718, https://doi.org/10.1093/bioinformatics/btw232 (2016).
46. Keller, A., Chavez, J. D., Eng, J. K., Thornton, Z. & Bruce, J. E. Tools for 3d interactome visualization. *Journal of Proteome Research* **18**, 753–758, https://doi.org/10.1021/acs.jproteome.8b00703 (2018).
47. Keller, A., Bakhtina, A. A., Chavez, J. D. & Bruce, J. E. Improved interpretation of protein conformational differences and ligand occupancy in large-scale cross-link data. *Journal of Proteome Research* **21**, 1475–1484, https://doi.org/10.1021/acs.jproteome.2c00109 (2022).
48. Manalastas-Cantos, K. et al. Modeling flexible protein structure with alphafold2 and crosslinking mass spectrometry. *Molecular & Cellular Proteomics* **23**, 100724, https://doi.org/10.1016/j.mcpro.2024.100724 (2024).
49. Schiffrin, B., Radford, S. E., Brockwell, D. J. & Calabrese, A. N. Pyxlinkviewer: A flexible tool for visualization of protein chemical crosslinking data within the <scp>pymol</scp> molecular graphics system. *Protein Science* **29**, 1851–1857, https://doi.org/10.1002/pro.3902 (2020).
50. Lagerwaard, I. M., Albanese, P., Jankevics, A. & Scheltema, R. A. Xlink mapping and analysis (xmas) - smooth integrative modeling in chimerax. *bioRxiv*https://doi.org/10.1101/2022.04.21.489026 (2022).
51. Matzinger, M. et al. Mimicked synthetic ribosomal protein complex for benchmarking crosslinking mass spectrometry workflows. *Nat. Commun.* **13**, https://doi.org/10.1038/s41467-022-31701-w (2022).
52. Szklarczyk, D. et al. The string database in 2023: protein–protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Research* **51**, D638–D646, https://doi.org/10.1093/nar/gkac1000 (2022).
53. LeDuc, R. D. et al. Proteomics standards initiative's proforma 2.0: Unifying the encoding of proteoforms and peptidoforms. *Journal of Proteome Research* **21**, 1189–1195, https://doi.org/10.1021/acs.jproteome.1c00771 (2022).
54. Perez-Riverol, Y. et al. The pride database resources in 2022: a hub for mass spectrometry-based proteomics evidences. *Nucleic Acids Research* **50**, D543–D552, https://doi.org/10.1093/nar/gkab1038 (2021).
55. Burke, A. M. et al. Synthesis of two new enrichable and ms-cleavable cross-linkers to define protein–protein interactions by mass spectrometry. *Organic & Biomolecular Chemistry* **13**, 5030–5037, https://doi.org/10.1039/c5ob00488h (2015).
56. Beveridge, R., Stadlmann, J., Penninger, J. M. & Mechtler, K. A synthetic peptide library for benchmarking crosslinking-mass spectrometry search engines for proteins and protein complexes. *Nat. Commun.* **11**, https://doi.org/10.1038/s41467-020-14608-2 (2020).
57. Jumper, J. et al. Highly accurate protein structure prediction with alphafold. *Nature* **596**, 583–589, https://doi.org/10.1038/s41586-021-03819-2 (2021).
58. Jiao, F. et al. Dsbso-based xl-ms analysis of breast cancer pdx tissues to delineate protein interaction network in clinical samples. *Journal of Proteome Research* **23**, 3269–3279, https://doi.org/10.1021/acs.jproteome.3c00832 (2024).
59. Shannon, P. et al. Cytoscape: A software environment for integrated models of biomolecular interaction networks. *Genome Research* **13**, 2498–2504, https://doi.org/10.1101/gr.1239303 (2003).
60. pandas development team, T. pandas-dev/pandas: Pandas, https://doi.org/10.5281/zenodo.3509134 (2020).
61. Wes McKinney. Data Structures for Statistical Computing in Python. In Stéfan van der Walt & Jarrod Millman (eds.) *Proceedings of the 9th Python in Science Conference*, 56 – 61, https://doi.org/10.25080/Majora-92bf1922-00a (2010).
62. Goloborodko, A. A., Levitsky, L. I., Ivanov, M. V. & Gorshkov, M. V. Pyteomics—a python framework for exploratory data analysis and rapid software prototyping in proteomics. *Journal of the American Society for Mass Spectrometry* **24**, 301–304, https://doi.org/10.1007/s13361-012-0516-6 (2013).
63. Levitsky, L. I., Klein, J. A., Ivanov, M. V. & Gorshkov, M. V. Pyteomics 4.0: Five years of development of a python proteomics framework. *Journal of Proteome Research* **18**, 709–714, https://doi.org/10.1021/acs.jproteome.8b00717 (2018).
64. Cock, P. J. A. et al. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics* **25**, 1422–1423, https://doi.org/10.1093/bioinformatics/btp163 (2009).
65. Hunter, J. D. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering* **9**, 90–95, https://doi.org/10.1109/MCSE.2007.55 (2007).
66. Raschka, S. Biopandas: Working with molecular structures in pandas dataframes. *J. Open Source Software***2**, https://doi.org/10.21105/joss.00279 (2017).
67. Deutsch, E. W. et al. The proteomexchange consortium at 10 years: 2023 update. *Nucleic Acids Research* **51**, D1539–D1548, https://doi.org/10.1093/nar/gkac1040 (2022).
68. Mirdita, M. et al. Colabfold: making protein folding accessible to all. *Nature Methods* **19**, 679–682, https://doi.org/10.1038/s41592-022-01488-1 (2022).
69. Mirdita, M., Steinegger, M. & Söding, J. Mmseqs2 desktop and local web server app for fast, interactive sequence searches. *Bioinformatics* **35**, 2856–2858, https://doi.org/10.1093/bioinformatics/bty1057 (2019).

Acknowledgements

We thank Martina A. Höllwarth, Eva Rauch, and Dennis Dannecker who gave feedback and tested pyXLMS. We also thank Melanie E. Birklbauer who designed the logo for pyXLMS.

Author contributions

M.J.B. conceptualized, implemented, tested and documented pyXLMS, performed data analysis and visualisation, and wrote the manuscript. L.M.B. and S.K. implemented, documented and tested pyXLMS. F.M. and M.M. conceptualized and tested pyXLMS, provided data, and helped with data analysis. K.M., S. W., and V.D. supervised the project. All

authors revised the manuscript and have given approval to the final version of the manuscript.

Funding

This work was supported by the F&E Infrastrukturförderung 4. Ausschreibung 2022/01 (AT-SCP, https://projekte.ffg.at/projekt/4795911, accessed on 24 November 2025) of the Austrian Research Promotion Agency (FFG). This work was further funded by the project LS20-079 of the Vienna Science and Technology Fund and the project P35045-B of the Austrian Science Fund (FWF, Grant DOI 10.55776/P35045), the project PAT2059025 of the Austrian Science Fund (FWF, Grant DOI 10.55776/PAT2059025), and the the ESPRIT program project number ESP 566 (Grant-DOI 10.55776/ESP566). Additionally, this work was supported by the MSCA-ITN-2020 PROTrEIN project that has received funding from the European Union's Horizon 2020 programme under the Marie Skłodowska-Curie grant agreement No. 956148. This research was funded in whole, or in part, by the Austrian Science Fund (FWF). For the purpose of open access, the author has applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41467-026-77407-1.

Correspondence and requests for materials should be addressed to Micha J. Birklbauer or Viktoria Dorfer.

Peer review information : *Nature Communications* thanks Diogo Borges and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at http://www.nature.com/reprints

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

© The Author(s) 2026